FUNDAMENTALS OF PROGRAMMING

Lecture 12: STRINGS & FILE HANDLING

Instructor: Dr Aqib Perwaiz

DEPARTMENT OF COMPUTER ENGINEERING
COLLEGE OF E&ME

STRINGS & FILE HANDLING

A Word About Character Arrays

- Character arrays enable us to work with words/phrases
  - Input terminates as soon as a whitespace character is entered
- Different syntax needed to make it work with whitespaces

cin.getline(array_name, array_length, delimeter)

cin.getline(my_char_array, my_length, '\n')

Strings

- What if we want to add other functionality on top of character arrays?
- We can use the string library in C++ (two options available):

  #include<cstring> (C-style strings. Offers a number of handy functions)

  #include<string> (Simplifies the usage of strings even further)
- String libraries allow us to manipulate strings in a handful of ways
  - Compare strings
  - Search strings for characters and other strings
  - Tokenize strings (separate strings into logical pieces)
- cstring built-in functions (methods) also work with character arrays

cstring vs string methods

Functionality	cstring	string
Copies the string **s2** into the character array **s1**. The value of **s1** is returned.	`char *strcpy( char *s1, const char *s2 );`	s1 = s2
Copies at most **n** characters of the string **s2** into the character array **s1**. The value of **s1** is returned.	`char *strncpy( char *s1, const char *s2, size_t n );`	s1 = s2.substr(start_pos, number_of_char)
Appends the string **s2** to the string **s1**. The first character of **s2** overwrites the terminating null character of **s1**. The value of **s1** is returned.	`char *strcat( char *s1, const char *s2 );`	s3 = s1+s2

cstring vs string methods

Functionality	cstring	string
Appends at most **n** characters of string **s2** to string **s1**. The first character of **s2** overwrites the terminating null character of **s1**. The value of **s1** is returned.	`char *strncat( char *s1, const char *s2, size_t n );`	s2 = s1+ s2.substr(start_pos, number_of_char)
Compares the string **s1** with the string **s2**. The function returns a value of zero, less than zero or greater than zero if **s1** is equal to, less than or greater than **s2**, respectively.	`int strcmp( const char *s1, const char *s2 );`	s1 == s2 s1 < s2 s1 >= s2
Find out the length of a string	`int strlen(const char * s1);`	s1.length()

Example

```
1
2  // Using strcpy and strncpy.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  #include <cstring>   // prototypes for strcpy and strncpy
9
10 int main()
11 {
12    char x[] = "Happy Birthday to You";
13    char y[ 25 ];
14    char z[ 15 ];
15
16    strcpy( y, x );  // copy contents of x into y
17
18    cout << "The string in array x is: " << x
19         << "\nThe string in array y is: " << y << '\n';
20
21    // copy first 14 characters of x into z
22    strncpy( z, x, 14 );  // does not copy null character
23    z[ 14 ] = '\0';       // append '\0' to z's contents
24
25    cout << "The string in array z is: " << z << endl;
```

Example (Cont'd)

```
26
27     return 0;  // indicates successful termination
28
29  } // end main
```

```
The string in array x is: Happy Birthday to You
The string in array y is: Happy Birthday to You
The string in array z is: Happy Birthday
```

- For practice, replicate the above function by using string methods

FILE HANDLING

File Handling

- So far:
  - In all programs, we have taken input from keyboard
  - Manipulated it in some way and displayed it on the monitor
- As soon as the programme terminates, all our variables/data lost
- Can be difficult to enter large amount of input data
- The solution: use File Handling

Using Input/Output Files

- Files in C++ are interpreted as a sequence of bytes stored on some storage media.
- The data of a file is stored in either readable form or in binary code called as text file or binary file.
- The flow of data from any source to a sink is called as a stream
- Computer programs are associated to work with files as it helps in storing data & information permanently.
- File - itself a bunch of bytes stored on some storage devices.
- In C++ this is achieved through a component header file called fstream.h
- The I/O library manages two aspects- as interface and for transfer of data.
- The library predefine a set of operations for all file related handling through certain classes.

Using Input/Output Files

A computer file

- is stored on a secondary storage device (e.g., disk);
- is permanent;
- can be used to provide input data to a program or receive output data from a program, or both;
- must be opened before it is used.

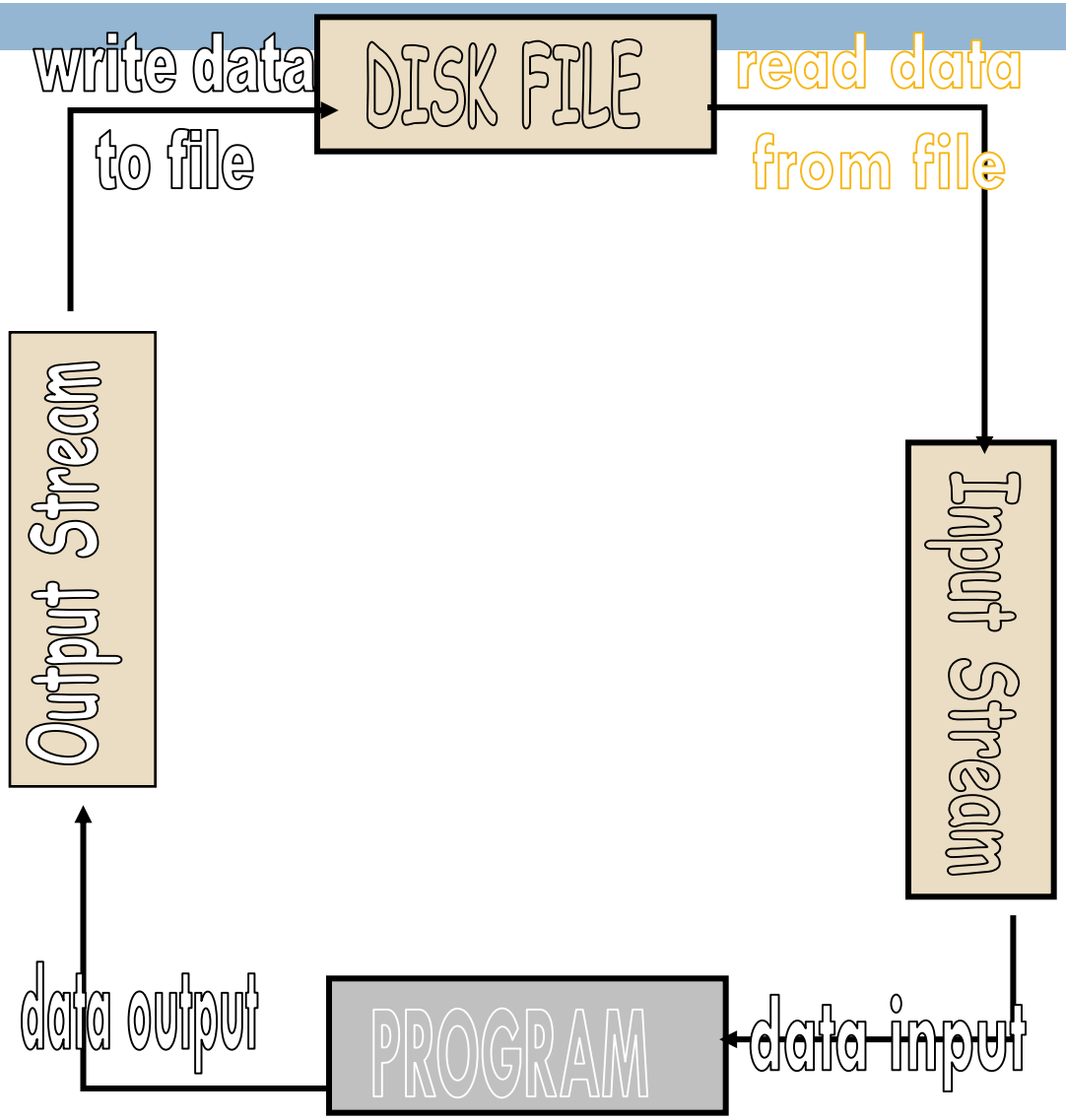
General File I/O Steps

- Declare a file name variable
- Associate the file name variable with the disk file name
- Open the file
- Use the file
- Close the file

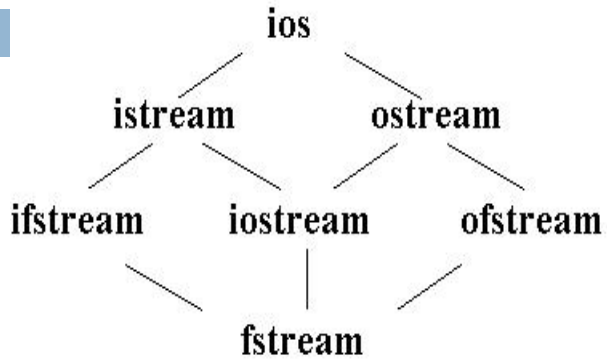
Using Input/Output Files

- Streams act as an interface between files and programs. In C++ . A stream is used to refer to the flow of data from a particular device to the program's variablesThe device here refers to files, keyboard, console, memory arrays. In C++ these streams are treated as objects to support consistent access interface.
- They represent as a sequence of bytes and deals with the flow of data.
- Every stream is associated with a class having member functions and operations for a particular kind of data flow.
- File -> Program (Input stream) - reads
- Program -> File (Output stream) – write

- All designed into fstream.h and hence needs to be included in all file handling programs.
- Diagrammatically as shown in next slide


write data
to file
DISK FILE
read data
from file
Output Stream
Input Stream
data output
PROGRAM
data input

Classes for Stream I/O in C++



ios is the base class.
istream and ostream inherit from ios
ifstream inherits from istream (and ios)
ofstream inherits from ostream (and ios)
iostream inherits from istream and ostream (& ios)
fstream inherits from ifstream, iostream, and ofstream

When working with files in C++, the following classes can be used:
ofstream – writing to a file
ifstream – reading for a file
fstream – reading / writing

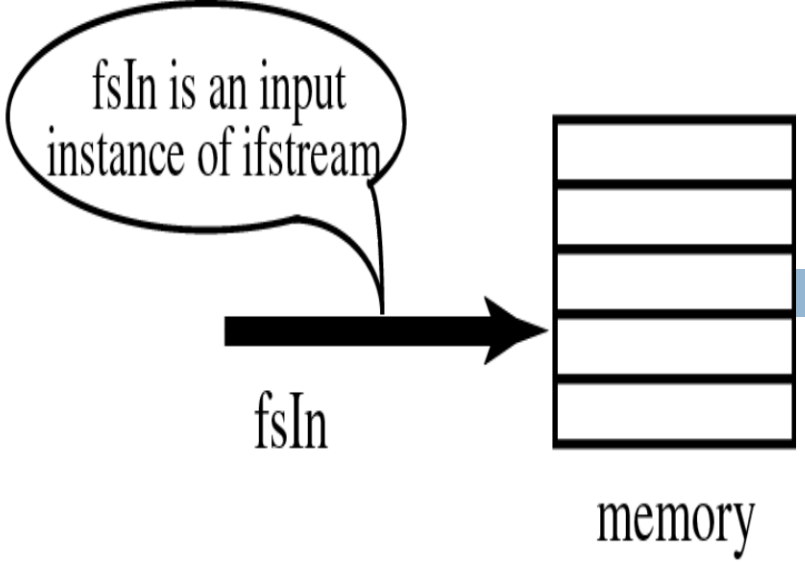
When ever we include <iostream.h>, an ostream object, is automatically defined – this object is cout.
ofstream inherits from the class ostream (standard output class).
ostream overloaded the operator >> for standard output….thus an ofstream object can use methods and operators defined in ostream.

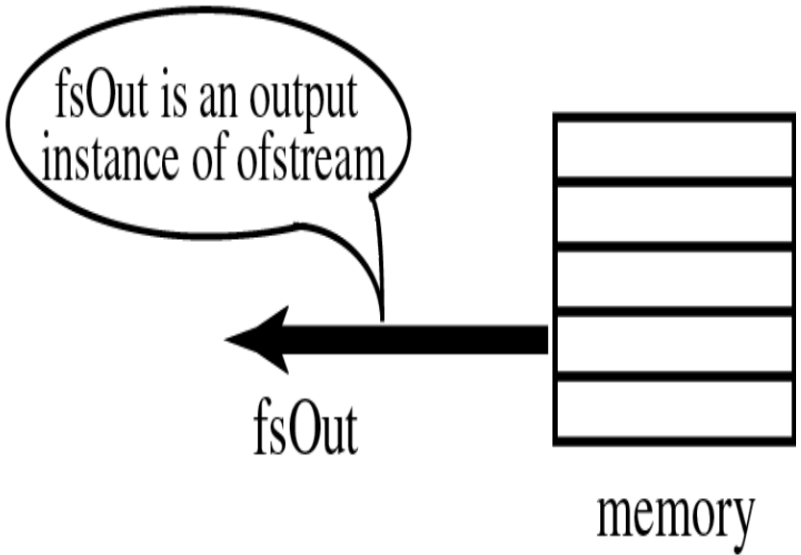
```
#include <fstream.h>

int main (void)
{
// Local Declarations
   ifstream     fsIn;

   ofstream     fsOut;
   .
   .
   .

}  // main
```





Using Input/Output Files

- ***stream*** - a sequence of characters
  - interactive (iostream)
    - **cin** - input stream associated with **keyboard.**
    - **cout** - output stream associated with **display.**
  - file (fstream)
    - **ifstream** - defines new input stream (normally associated with a file).
    - **ofstream** - defines new output stream (normally associated with a file).

• Stream of bytes to do input and output to different devices.
• Stream is the basic concepts which can be attached to files, strings, console and other devices.
• User can also create their own stream to cater specific device or user defined class.

File Handling

- File handling allows us to permanently store the data in text or binary files

- We use certain libraries to write/read data to a file

  #include <fstream> (This is what you will be using)

  #include <ofstream>

  #include <ifstream>

File Handling

- To write/read from a file, we need to:
  - Create or open a file
  - Write/Read the data to/from the file
  - Close the file after our operation

```cpp
#include <iostream>
#include <string>
#include <fstream>

using namespace std;

int main()
{
   char str[80];
string s1 = "Fortis Fortuna Aduivat";

//cin>>str;
cin.getline(str,100);

//Create a file with certain name and extension. Outfil is handle of the flie
ofstream outfl("storage.txt");

outfl<<str; //writes char arr str
outfl<<s1; //write string s1

outfl<<'\0';

outfl.close();

ifstream  obj("storage.txt");  //handle for input file
obj.getline(str,100);
cout<<"File contents"<<str;
obj.close();
return 0;
}
```

Acknowledgements

1. Deitel and Deitel: C++ How to Program, 7th Edition, Prentice Hall Publications
2. Robert Lafore: Object-Oriented Programming in C++, Fourth Edition, December 2001,Sams Publishing .
3. A Structured Programing Approach Using C++ by Behrouz A. Forouzan
4. www.cplusplus.com

Material in these slides has been taken from, the following resources